

The *s-matches-p* function in the *s.el* library takes a regular expression and a string, and returns 'true' if there is a match and 'nil' otherwise. If an optional *START* argument is provided to it, then it starts the search from that index.

```
(s-matches-p REGEXP S &optional START)
;; Syntax
```

You can also use the *in* construct to generate the same '[0-9A-Fa-f]' regular expression pattern as illustrated below:

```
(in SET ...) ;; Syntax

(s-matches-p (rx (in "a-f" "A-F" "0-9"))
  "A")
t
```

The *and* construct can be used to combine multiple s-expressions together. In the following example, *line-start* generates the '^' (caret) character and is combined with zero or more occurrences of alphabets as shown below:

```
(and SEXP1 SEXP2 ...) ;; Syntax

(rx (and line-start (0+ (in "a-z"))))
"^[a-z]*"
```

If you want to match either of the s-expressions, you can use the *or* construct as illustrated below:

```
(or SEXP1 SEXP2 ...) ;; Syntax

(s-match-strings-all
  (rx (or "Mary" "Peter" "John"))
  "Mary had a little lamb")
(("Mary"))
```

The *s-match-strings-all* function from the *s.el* library takes two arguments: a REGEX and a STRING. It returns a list of matches for REGEX in the given input STRING.

```
(s-match-strings-all REGEX STRING) ;;
```

Syntax

The *not* rx-constituent can be used to generate a negative match. In the following example, a regular expression is generated to match a newline followed by anything that is not a blank.

```
(not SEXP) ;; Syntax

(rx (and "\n" (not blank)))
"\n[^\n:blank:]"
```

You can match for a specific character literally using the *char* construct. A couple of examples to match for the character ';' are given below:

```
(rx (char ";"))
";"

(s-matches-p (rx (char ";"))
  "ABC")
nil

(s-matches-p (rx (char ";"))
  "ABC;")
t
```

The negate operation to not match a character is provided by the 'not-char' construct. For example:

```
(not-char SEXP1 SEXP2 ...) ;; Syntax

(rx (not-char "A"))
"[^A]"

(s-matches-p (rx (not-char "A"))
  "B")
t

(s-matches-p (rx (not-char "A"))
  "A")
nil
```

The actual code for *not-char* in *rx.el* is defined as follows:

```
(defconst rx-constituents
  '(((...
    (not-char . (rx-not-char 1 nil rx-
      check-any))
```

```
...)))
```

```
(defun rx-not-char (form)
  "Parse and produce code from FORM.
FORM is `(not-char ...)'."
  (rx-check form)
  (rx-not `(not (in ,@(cdr form))))))
```

You can generate regular expressions from s-expressions to match for zero or more, one or more, and zero or one occurrences, as illustrated below:

```
(zero-or-more SEXP ...) ;; Syntax
(one-or-more SEXP ...) ;; Syntax
(zero-or-one SEXP ...) ;; Syntax

(rx (zero-or-more "x"))
"x*"

(s-matches-p (rx (zero-or-more "x"))
  "yz")
t

(s-matches-p (rx (zero-or-more "x"))
  "xyz")
t

(rx (one-or-more "x"))
"x+"

(s-matches-p (rx (one-or-more "x"))
  "yz")
nil

(s-matches-p (rx (one-or-more "x"))
  "xyz")
t

(rx (zero-or-one "x"))
"x?"

(s-matches-p (rx (zero-or-one "x"))
  "yz")
t

(s-matches-p (rx (zero-or-one "x"))
  "xyz")
t
```

We have already seen the *line-start* construct that generates the caret symbol (^). Similarly, you can use the *line-end* construct to signify the end of